The set $X^* = \{(x_i^*, t_i^*)\}_{i=1}^M$ is the set of training points, and M is the number of training points. We highlight here that, since we are going to compare the PINN solution to synthetic data, we select collocation points coincident with training points. We recall that training points are used to teach the network to fit the known solution in the data-driven regions of the problem space, whereas collocation points are used to ensure that the solution, provided by the neural network, respects the physical law modelled by the differential equation considered (for further details we refer to [47]). Therefore, in general training points and collocation points could be different collocation points are a subset of training points but, here, they will be given by the same set of points.

The chosen norm $\|\cdot\|$ (it may be different for each term in the loss function) depends on the functional space and the specific problem. Selecting a correct norm (to avoid overfitting) for the loss function evaluation is an important problem in PINN, and recently, in [48], the authors have proposed spectral techniques based on Fourier residual method to overcome computational and accuracy issues. The first term in the right-hand side of Eq. (6) is referred to as data fitting loss and could possibly handle both initial and boundary conditions, while the second term is referred to as residual loss, which is responsible for making the NN informed by the physics of the problem. The derivatives inside $\mathcal{D}$ in space, time and in the parameter space are usually performed using `autodiff` (Automatic Differentiation algorithm, see [49,50]). Using the NN to approximate u in the loss function Eq. (6) allows us to solve the PDE by minimising the loss function with respect to the parameters θ of the NN. If $u \approx u_{NN}(\theta)$, then the minimisation problem can be written as

$$\theta^\dagger = \arg\min_\theta \mathcal{L}(u_{NN}(\theta); \theta_0). \tag{7}$$

For a more detailed discussion on the PINN structure and the loss function, we refer to [51], [7] and to the review in [52].

3.1. Inverse PINN

Inverse PINNs are a type of Neural Network specifically designed to determine constitutive parameters or problem-related functions that appear in the PDE one must solve. However, due to the limited amount of data relative to exact solutions, or of available measurements of the physical problem described by the PDE underlying the PINN, the inverse problem (7) could likely be ill-posed, and thus particular care has to be put into the training strategy during the optimisation process (see, e.g., [53]). In particular, different contributions in the loss functions Eq. (6) could conflict with each other, providing an unbalanced gradient back-propagation during the training, which would result in a troublesome convergence process [54]. Thus, several strategies have been recently developed to cope with these issues: among the others, one could resort to *GradNorm* [55] to dynamically tune gradient magnitudes to balance learning tasks; to *PCGrad* [56] to project each gradient on the tangent plane to all the other conflicting gradients to mitigate such destructive interference; to *Multi-Objective Optimisation* [57]; to *Self-Adaptive PINNs* [58], where each training point is weighed individually, so to penalise more points in difficult regions of the domain.

Using the notation introduced in the previous section, the inverse PINN minimisation now takes into account also the physical parameters and can be written as

$$(\theta^\dagger, \theta_0^\dagger) = \left[\arg\min_{\theta,\theta_0} \mathcal{L}(u_{NN}(\theta); \theta_0) + \iota\|\theta_0 - \theta_0^*\|^2\right], \tag{8}$$

where ι is a regularisation parameter. The second term in the right-hand side of the equation is the regularisation term, which is used to prevent overfitting and to ensure that the physical parameters θ_0 are close to the some reference parameters θ_0^*. If otherwise stated, in the following we will consider $\iota = 0$.

With reference to the mathematical models introduced in Section 2, we will consider the following physical parameters to be estimated: the diffusion coefficient D in the heat equation (1), the velocity V and the dispersion coefficient D in the advection–diffusion–reaction equation (3), and the transfer coefficient λ in the mobile–immobile model (4). The physical parameters will be considered as trainable parameters in the NN, and the reference data will be added to the loss function Eq. (6).

3.2. Adaptive inverse PINN

To ensure the convergence of the inverse PINN, we redefine a weighted loss function as

$$\mathcal{L}(u_{NN}(\theta); \theta_0) = \sum_{i=1}^{M} \left(\omega_i^k \|u(x_i^*, t_i^*) - u_i^*\|^2 + \omega_D^k \|\mathcal{D}(u(x_i^*, t_i^*); \theta_0)\|^2\right), \tag{9}$$

where ω_i^k are weight factors that depends on the training iteration i. The weights are updated at each iteration to ensure that the different components of the loss function are balanced. The weights are updated using the following formula:

$$\omega_i^k = \frac{\hat{\omega}_i^k}{\sum_{j=1}^M \hat{\omega}_j^k + \hat{\omega}_D^k}, \qquad i = 1, \dots, M, \tag{10}$$

$$\omega_D^k = \frac{\hat{\omega}_D^k}{\sum_{j=1}^M \hat{\omega}_j^k + \hat{\omega}_D^k}, \tag{11}$$

$$\hat{\omega}_i^k = \begin{cases} \eta_{BC} & \text{if } x_i^* \in \partial\Omega \\ \eta_{IC} & \text{if } t_i^* = 0 \\ \nu(k)\eta_u & \text{if } (x_i^*, t_i^*) \text{ is a collocation point} \\ 0 & \text{otherwise} \end{cases}, \qquad i = 1, \dots, M, \tag{12}$$

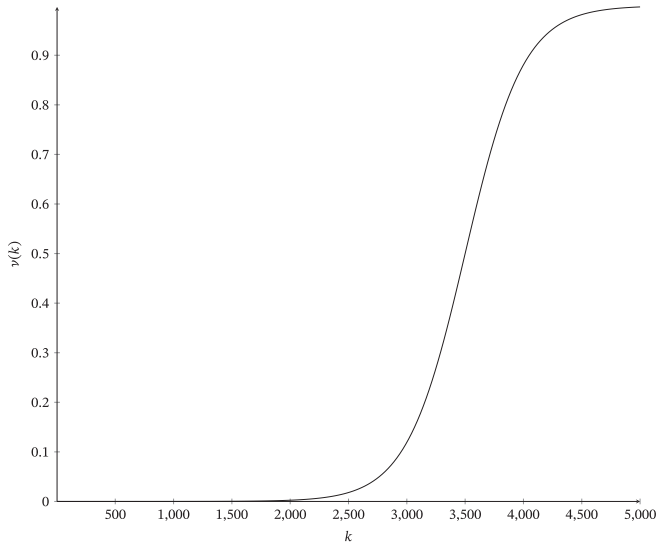

Fig. 3. Qualitative behaviour of ν_k in (14) for $K = 5000$ and $K_0 = 1000$.

$$\hat{\omega}_D^k = 1, \tag{13}$$

where η_{BC}, η_{IC}, and η_u are the weights for the boundary conditions, initial conditions, and collocation points, respectively. The function ν_k is an increasing function of the epoch k, such that $\nu_k = 0$ and $\nu_k \to 1$ as $k \to \infty$. This allows the PINN to be trained initially solely by the PDE residual. In the following, we will consider

$$\nu(k) = \frac{\tanh\left(10\left(\frac{k-K/2-K_0}{K}\right)\right)+1}{2}, \qquad k = 1, \dots, K, \tag{14}$$

where K is the total number of epochs, and K_0 is a threshold epoch before the weights are updated more significantly; see Fig. 3 for a typical graph of a function of this kind.

The gradients $\nabla_\theta \mathcal{L}$ and $\nabla_{\theta_0} \mathcal{L}$ are computed with the `autodiff` algorithm, and the latter (the gradients with respect to the physical parameters) are scaled by $\gamma\nu(k)$ at each iteration. The scaling of the gradients is crucial for the convergence of the inverse PINN, as it ensures that the physical parameters are updated only when data is included in the loss function. The parameters are then updated with the Adam optimiser, with a sequence of learning rates that decrease at each iteration according to the epoch k. Namely starting from a learning rate α_0 at the first epoch, the learning rate is updated as:

$$\alpha_k = \alpha_0 \beta^{\left\lfloor \frac{k}{100} \right\rfloor}, \tag{15}$$

where $0.9 < \beta < 0.99$ is a constant factor. An algorithmic description of the above process is given in Algorithm 1.

Algorithm 1 Training Algorithm with Adaptive Weights and Gradient Updates.

```
epoch = 0
repeat
    epoch = epoch + 1
    if do_parameter_train and epoch > K_0 then
        compute ν(epoch) as in Eq. (14)
    end if
    update data weights as in Eq. (10)
    compute gradients of loss function
    rescale gradients relative to θ_0 by ν(epoch)
    apply gradients to all trainable parameters
until convergence or epoch > epochs
```

4. Numerical results

In this section, we apply our PINN to different models arising from Eq. (3) and Eq. (4), under several assumptions and conditions. All codes and data used in this work are published and freely available [39]. We report here a series of numerical experiments starting from a random initial guess for the parameters, and we show the convergence of the PINN to the correct values. The robustness of the approach with respect to the initial value of the parameters is shown in Appendix through two additional random initial